

Groupon Deal Page Optimizer – Project Write-Up

What I Built

A three-stage Python pipeline (Audit → Research → Propose) where Claude is used differently at each stage rather than once at the end.

Stage 1 – Audit. Playwright fetches the rendered page. A layered parser stack extracts data in priority order: JSON-LD ProductGroup schema first, then Groupon's `__NEXT_DATA__` hydration blob, then HTML selectors as fallback. Claude scores the page on six weighted dimensions using extended thinking, specifically so it reasons about what's *absent* rather than what's present – identifying missing appointment policies or unverifiable discount claims requires reasoning about customer intent, not extraction.

Stage 2 – Research. Four async tasks run in parallel: Yelp scraping, Google Business extraction, category-aware competitor search, and market benchmarking. The competitor finder uses 20 category-specific query templates ("juice cleanse price Milwaukee" rather than a generic service query) and validates every result against a domain blocklist (Investopedia, BLS, tourism boards rejected). Every competitor record carries a `sku_match_confidence` score, and Claude is explicitly told: no dollar comparison claims if all confidence scores are below 40%.

Stage 3 – Proposal. Claude receives all four data sources simultaneously and scores recommendations by `impact_score = visibility × user_importance × evidence_strength`. This ensures shipping transparency (score ~800) ranks above address clarification (~60). Every recommendation requires `supporting_evidence` – a review theme with mention count, a competitor price with confidence level, or a specific audit score. The schema rejects generic claims.

Execution is checkpointed via a `pipeline_runs` DuckDB table – re-runs skip completed stages, so iterating on a prompt doesn't mean re-scraping or re-paying for API calls.

What I Would Improve With More Time

The biggest gap is **third-party review data**. Review themes are the most differentiated output – "Yelp mentions wait times 23 times but the deal page says nothing about appointment requirements" is far stronger than generic copy advice – but Yelp returns 403s consistently. Better Playwright-based scraping or a reviews API would unlock that.

Second: **a validation pass on proposals**. The architecture specifies generate → validate citations → re-rank, but in practice it runs single-pass. A second Claude call that checks whether each recommendation's `data_citation` actually appears in the research context would catch hallucinated claims before they reach the report.

Third: **an executive dashboard** querying DuckDB across all 20 deals. The schema supports "rank deals by value_comm gap" or "show all trust_score < 4" but the UI to surface that doesn't exist yet.

What Surprised Me During the Research

How often the original price simply isn't there. Groupon shows a discount percentage prominently, but the reference price is frequently absent from the JSON-LD schema and the rendered HTML – it exists in the React UI as a strikethrough but isn't machine-readable without specifically hunting for `<s>` tags. This became a top recommendation across almost every category: the savings claim is unverifiable by design.

How much category signal matters for research quality. Searching for competitors for an "online acupressure diploma" kept returning massage service providers, not e-learning platforms. The query was structurally correct but semantically wrong, cascading into useless category context data. Fixing it required building a 20-category detection system that fires before the first search query.

Fine print is where conversions are lost. Parking, hours, cancellation terms, what to bring, whether appointment is required – almost none of the deal pages had all of them, and they were the most consistent content gaps across all 20 deals and all four categories. A cautious buyer asking "but what do I actually do after I buy this?" rarely gets an answer.